

## AiSD Lista 2

Dominik Kowalczyk

11 stycznia 2025

### 1 Algorytm sortowania QUICKSORT (sortowanie szybkie)

Do prawidłowej implementacji algorytmu QUICKSORT potrzebna jest funkcja pomocnicza PARTITION, oraz funkcja główna QUICKSORT. W podstawowej wersji kod wygląda następująco:

Listing 1: Kod funkcji sortowania szybkiego

```
1 int PARTITION(float A[] , int p, int r) {  
2     float x = A[r];  
3     int i = p - 1;  
4  
5     for (int j = p; j <= r - 1; j++) {  
6         if (A[j] <= x) {  
7             i = i + 1;  
8             float temp = A[i];  
9             A[i] = A[j];  
10            A[j] = temp;  
11        }  
12    }  
13    float temp = A[i + 1];  
14    A[i + 1] = A[r];  
15    A[r] = temp;  
16    return i + 1;  
17 }
```

W funkcji Partition jako argumenty przyjmujemy tablicę liczb zmiennoprzecinkowych oraz indeksy początku p oraz końca r. Używamy pivota, którym jest nasz  $x$  któremu przypisujemy wartość

ostatnią z tablicy ( $A[r]$ ). Ustawiamy również indeks  $i$ , który będzie rozdzielał części posortowane od nieposortowanej. Następnie przechodzimy do pętli:

- **for (int j = p; j <= r - 1; j++)**  
Inicjuje pętlę, która iteruje od pierwszego elementu do  $r - 1$  tablicy  $A[1]$ .
- **if (A[j] <= x)**  
Sprawdzamy, czy nasz pivot jest większy od aktualnie sprawdzanej wartości. Jeśli jest to przesuwamy indeks naszej granicy o jeden w prawo  $i = i + 1$  oraz zamieniamy naszą badany element z następnym w kolejności (w kodzie używamy zmiennej tymczasowej, aby nie utracić naszego elementu).

Gdy, pętla przestanie działać, czyli pozostałym elementom są większe od naszego aktualnego pivotu, otrzymujemy podział na dwie podtablice a nasz pivot zamieniamy z elementem o indeksie za naszą granicą.

Listing 2: Kod funkcji sortowania szybkiego

```
1 void QUICK_SORT(float A[], int p, int r) {  
2     if (p < r) {  
3         int s = PARTITION(A, p, r);  
4         QUICK_SORT(A, p, s - 1);  
5         QUICK_SORT(A, s + 1, r);  
6     }  
7 }
```

We właściwej funkcji QUICKSORTA mamy pętlę if, w której z naszego podziału tablicy z funkcji pomocniczej PARTITION bierzemy  $i$  przypisujemy indeks  $s$  i dzielimy na jego podstawie na dwie podtablice, które sortujemy rekurencyjnie.

Modyfikacja do algorytmu QUICKSORT (sortowania szybkiego) polega na dzieleniu tablicy na trzy przy pomocy dwóch elementów (na 3 części).

Listing 3: Kod funkcji sortowania szybkiego z modyfikacją

```

1  float p1 = A[lewy];
2  float p2 = A[prawy];
3
4  if (p1 > p2) {
5      swap(A[lewy], A[prawy]);
6      p1 = A[lewy];
7      p2 = A[prawy];
8  }
9
10 int L = lewy + 1;
11 int S = lewy + 1;
12 int P = prawy - 1;

```

W przypadku modyfikacji mamy dwa pivoty  $p1$  oraz  $p2$  gdzie *lewy* oznacza pierwszy element tablicy (z lewej strony) a *prawy* ostatni. Następnie mamy pętlę porównującą wartości pivotów i jeśli jest taka potrzeba zamienia je miejscami (większy z nich - na końcu tablicy, mniejszy na początku).

Mamy również trzy nowe zmienne  $L$  (element drugi (pierwszy rozpatrywany po pominięciu pivota)),  $S$  (aktualnie rozpatrywany element) oraz  $P$  (element przedostatni (ostatni przy pominięciu pivota)).

Listing 4: Kod funkcji sortowania szybkiego z modyfikacją

```

1  while (S <= P) {
2      if (A[S] < p1) {
3          swap(A[S], A[L]);
4          L++;
5      } else if (A[S] >= p1 && A[S] <= p2) {
6      } else {
7          while (A[P] > p2 && S < P)
8              P--;
9          swap(A[S], A[P]);
10         P--;

```

```

11 |                 if (A[S] < p1) {
12 |                     swap(A[S], A[L]);
13 |                     L++;
14 |                 }
15 |             }
16 |             S++;
17 | }

```

W powyższym kodzie mamy parę kluczowych pętli:

- **if (A[S] < p1)**  
Jeśli nasz obecnie sprawdzany element jest mniejszy od naszego pierwszego pivota to *swap(A[S], A[L]);* - zamienimy miejscami nasz aktualny element z pierwszym elementem w tablicy i przesuwamy naszą granicę w prawo (*L++*).
- **else if (A[S] >= p1 "and" A[S] <= p2)**  
Wartość naszego sprawdzanego elementu jest pomiędzy wartością *p1* a *p2*, zatem jest we właściwym miejscu - nic nie robimy. (linijka zawarta w kodzie głównie, aby zliczać porównanie).
- **while (A[P] > p2 "and" S < P)**  
Tutaj rozpatrujemy przypadek, w którym nasz badany element jest większy od drugiego pivota. Wtedy (za sprawą pętli *while*) szukamy elementu, który jest mniejszy od większego z pivotów i przesuwamy go do środkowej części, zmniejszając prawą granicę w lewo. Zamieniamy rozpatrywany element z tym znalezionym z poprzedniej pętli i znowu idziemy z prawą granicą w lewo. Jeśli nasz nowy rozpatrywany element jest mniejszy od naszego mniejszego z pivotów to przenosimy go do części dla elementów mniejszych od *p1* i idziemy z granicą lewą granicą w prawo.

Funkcja główna jest już mało ciekawa, gdyż różni się od wersji bez modyfikacji tylko tym, że mamy jeden dodatkowy obszar do rozważenia przy sortowaniu rekurencyjnym.

## 2 Algorytm sortowania RADIXSORT (z użyciem COUNTINGSORT)

Sortowanie pozycyjne jesteśmy w stanie zaimplementować w oparciu o, między innymi sortowanie przez zliczanie. Zaczniemy od przeanalizowania właśnie jego.

Listing 5: Kod funkcji sortowania pozycyjnego

```
1 void COUNTINGST(int A[], int n, int exp, int d){
2     int* B = new int[n];
3     int* C = new int[d]();
4
5     for (int i = 0; i < n; i++) {
6         int j = (A[i] / exp) % d;
7         C[j]++;
8     }
9
10    for (int i = 1; i < d; i++) {
11        C[i] += C[i - 1];
12    }
13
14    for (int i = n - 1; i >= 0; i--) {
15        int j = (A[i] / exp) % d;
16        B[C[j] - 1] = A[i];
17        C[j]--;
18    }
19
20    for (int i = 0; i < n; i++) {
21        A[i] = B[i];
22    }
23
24    delete [] B;
25    delete [] C;
26 }
```

W tym przypadku, jako kod analizę należałoby zacząć od wyjaśnienia czym jest `exp` (wykładnik) i `d` (podstawa systemu liczbowego). Tworzymy dwie tablice, jedna do przechowywania ilości zliczeń (w naszym wypadku to `C[]`) oraz wynikowa, czyli `B[]`. W pierwszej pętli *for* wyznaczamy cyfry setek, dziesiątek, jedności,

itd. i zapisujemy w tablicy zliczeń. W następnej funkcji sumujemy poprzednie wartości zliczeń, aby prawidłowo wyznaczyć miejsce przypisania do tablicy wynikowej. Umieszczamy wyniki z tablicy C do B (cofając indeks o jeden abyśmy zaczęli od początku B). Na sam koniec przypisujemy wypełnioną tablicę wynikową do tablicy właściwej  $A[]$ .

Listing 6: Kod funkcji sortowania pozycyjnego

```
1 void RADIX_SORT(int A[], int n, int d) {
2     int maks_war = A[0];
3     for (int i = 1; i < n; i++) {
4         if (A[i] > maks_war) {
5             maks_war = A[i];
6         }
7     }
8     int k = 0;
9     while (maks_war > 0) {
10        maks_war /= d;
11        k++;
12    }
13    for (int exp = 1, iter = 0; iter < k;
14        exp *= d, iter++) {
15        COUNTING_SORT(A, n, exp, d);
16    }
17 }
```

W funkcji RADIXSORT na dobrą sprawę niewiele się dzieje. Właściwe to większość funkcji jest wyznaczenie wartości  $k$ , która reprezentuje maksymalną liczbę cyfr w największej liczbie. Kod jednakowoż to nic innego jak znajdowanie największej liczby w tablicy i zliczenie jej ilości cyfr. Oczywiście w funkcji występuje również wywołanie sortowania przez zliczanie, które przy posiadaniu zmiennej, mamy granicę mówiącą jak długo mamy iterować po zmiennych.

Modyfikacja do RADIXSORTA polega na tym, aby funkcja sortowała liczby całkowite (czyli dodatnie i ujemne) a nie same dodatnie, tak jak mamy w przypadku klasycznego algorytmu.

Mój pomysł na implementację tej modyfikacji jest podzielenie na dwie tablice (oddzielne) liczb ujemnych i dodatnich, po czym dodat-

nie posortować w taki sam sposób jak w klasycznej wersji algorytmu, a dla ujemnych zmienić znak i również zastosować sprawdzony już kod, posortować odwrócić, przywrócić znak i złączyć obie podtablice. Ze względu na to, że kod jest długi i w większości analogiczny to przedstawię sposób uporządkowania liczb ujemnych.

Listing 7: Kod funkcji sortowania pozycyjnego z modyfikacją

```

1 void RADIX_SORT_MOD(int A[], int n, int d) {
2     int licz_dodatnich = 0;
3     int licz_ujemnych = 0;
4
5     for (int i = 0; i < n; i++) {
6         if (A[i] >= 0) {
7             licz_dodatnich++;
8         } else {
9             licz_ujemnych++;
10        }
11    }
12    int*dodatnie = new int[licz_dodatnich];
13    int*ujemne = new int[licz_ujemnych];
14
15    int plus = 0;
16    int minus = 0;
17
18    for (int i = 0; i < n; i++) {
19        if (A[i] >= 0) {
20            dodatnie[plus++] = A[i];
21        } else {
22            ujemne[minus++] = -A[i];
23        }
24    }
25 }

```

W powyższym kodzie, tak jak ostatnio nie mamy nic spektakularnego rozdzielamy na liczby dodatnie i ujemne, tworzymy dla nich dwie osobne tablice i przypisujemy odpowiednie elementy z uwagą, że przy przypisywaniu liczb ujemnych do tablicy zmieniamy ich znak na dodatni.

Listing 8: Kod funkcji sortowania pozycyjnego z modyfikacją

```

1  int max_ujemne = 0;
2  if (liczba_ujemnych > 0) {
3      max_ujemne = ujemne[0];
4  }
5  for (int i = 1; i < liczba_ujemnych; i++) {
6      if (ujemne[i] > max_ujemne) {
7          max_ujemne = ujemne[i];
8      }
9  }
10
11 int k_ujemne = 0;
12 while (max_ujemne > 0) {
13     max_ujemne /= d;
14     k_ujemne++;
15 }

```

W tym przypadku dostajemy bardzo analogiczny przypadek co w klasycznym RADIXSORCIE. Potrzebujemy określić ilość cyfr w największej i najmniejszej liczbie (bo 500 i -500 może się wydarzyć). Ponownie otrzymujemy wielokrotnie wątkowany w tym sprawozdaniu moment szukania wartości największej i jak wcześniej zliczania cyfr. Następnie sortujemy oddzielnie tablice dodatnie oraz ujemne, łączymy je (z zastrzeżeniem, że najpierw wpisujemy do tablicy właściwej cyfry z podtablicy dla liczb ujemnych ze zmianą znaku, a następnie dodatnie.).

### 3 Algorytm sortowanie kubełkowego BUCKET-SORT (z użyciem INSERTION SORT)

Algorytm BUCKET SORT bazuje swoje sortowanie na sortowaniu przez wstawianie zaimplementowane na listach. Przejdźmy najpierw może przez kod INSERTIONSORTA na listach.



### 3.1 INSERTIONSORT na listach

Listing 9: Kod funkcji sortowania przez wstawianie na listach

```
1 struct WEZEL {  
2     float wartosc;  
3     WEZEL* prev;  
4     WEZEL* next;
```

Zaczynamy od zainicjowania elementu listy - *WEZEL*, gdzie *wartosc* przechowuje wartość węzła (liczbę zmiennoprzecinkową). Wyróżniamy również *prev*, który wskazuje na węzeł poprzedni oraz *next* wskazujący na element następny.

Listing 10: Kod funkcji sortowania przez wstawianie na listach

```
1 struct LISTA {  
2     WEZEL* head;  
3     LISTA() : head(nullptr) {}
```

Definiujemy listę jako strukturę danych, do której inicjalizacji używamy "głowy" (*head*), która na początku przechowuje pierwszy element listy. Tworzymy pustą listę.

Następnie do struktury danych listy dodajemy funkcje określające jej działanie. W skład ich wchodzi funkcja usuwająca element, drukująca listę, czy szukająca elementu, jednak najciekawsza z nich jest funkcja dodająca element do listy.

Listing 11: Kod funkcji sortowania przez wstawianie na listach

```
1  
2 void LIST_INSERT(LISTA& lista, WEZEL* x) {  
3     x->next = lista.head;  
4     x->prev = nullptr;  
5     if (lista.head == nullptr) {  
6         lista.head = x;  
7     } else {  
8         lista.head->prev = x;  
9         lista.head = x;  
10    }  
11 }
```

Ustawiamy nowy element listy na po nim następujący (*next*), gdyż świeżo dodany element będzie na pierwszym miejscu. Nowy węzeł oczywiście nie ma poprzednika, stąd  $x \rightarrow prev = \text{nullptr}$ . Jeśli lista jest pusta ustawiamy "głowę" na  $x$  bo jest to nasz jedyny element. W przeciwnym wypadku ustawiamy poprzednik listy na nasz  $x$  w celu połączenia nowego węzła z listą i przestawiamy na niego "głowę" bo to naturalnie pierwszy element listy.

Listing 12: Kod funkcji sortowania przez wstawianie na listach

```

1  void LIST_INSERTION_SORT(LISTA& l) {
2      WEZEL* element = l.head->next;
3      while (element != nullptr) {
4          float key = element->wartosc;
5          WEZEL* prev = element->prev;
6          while (prev != nullptr &&
7                 prev->wartosc > key) {
8
9                 prev->next->wartosc =
10                prev->wartosc;
11
12                prev = prev->prev;
13            }
14
15            if (prev == nullptr) {
16                l.head->wartosc = key;
17            } else {
18                prev->next->
19                wartosc=key;
20            }
21
22            element = element->next;
23        }
24    }

```

Wiedząc, że lista nie jest pusta, możemy posortować ją przy pomocy powyższego sortowania przez wstawianie, które działa idealnie tak samo jak ten na tablicach, przy czym tam przesuwaliśmy indeksy, to przesuwamy o węzły. Podobnie mamy klucz : *key*, która przechowuje aktualna wartość naszego elementu. Rozpoczynamy od drugiego elementu i kierujemy się w lewo listy. Pętla while ma

za zadanie umiejscowienie na odpowiednie miejsce naszego klucza przesuwać węzły. Schemat działania pod tym względem identyczny, doskonale znany.

### 3.2 BUCKET SORT

Mając napisany INSERTION SORT na listach możemy bez problemu przejść do implementacji sortowania kubełkowego.

Listing 13: Kod funkcji sortowania kubełkowego

```
1 void BUCKETSORT(float* A, int n) {
2     LISTA* B = new LISTA[n];
3
4     for (int j = 0; j < n; j++) {
5         B[j] = LISTA();
6     }
7     for (int i = 0; i < n; i++) {
8         int in=static_cast<int>(n*A[i]);
9         WEZEL*nwezel = new WEZEL(A[i]);
10        B[in].LIST_INSERT(B[in], nwezel);
11    }
12    for (int j = 0; j < n; j++) {
13        B[j].LIST_INSERTION_SORT(B[j]);
14    }
15    int s = 0;
16    for (int j = 0; j < n; j++) {
17        WEZEL* elem = B[j].head;
18        while (elem != nullptr) {
19            A[s++] = elem->wartosc;
20            elem = elem->next;
21        }
22    }
23    delete[] B;
24 }
```

Tworzymy tablicę pomocniczą B, która przechowuje początkowo puste listy.

- Operacje w pierwszej pętli for:

- Obliczenie indeksu koszyka:

```
int in = static_cast<int>(n * A[i]);
```

- Utworzenie nowego węzła z wartością:

```
WEZEL* nwezel = new WEZEL(A[i]);
```

- Dodanie elementu do odpowiedniego koszyka:

```
B[in].LIST_INSERT(B[in], nwezel);
```

- Sortowanie list w każdym koszyku:

- Iteracja po koszykach: `for (int j = 0; j < n; j++)`
- Sortowanie listy w koszyku:

```
B[j].LIST_INSERTION_SORT(B[j]);
```

- Przenoszenie posortowanych elementów z koszyków do tablicy A:

- Inicjalizacja indeksu:

```
int s = 0;
```

- Iteracja po koszykach: `for (int j = 0; j < n; j++)`
- Iteracja po elementach w koszyku:

- \* Pobranie pierwszego elementu:

```
WEZEL* elem = B[j].head;
```

- \* Przechodzenie przez listę:

- Przypisanie wartości z elementu do A:

```
A[s++] = elem->wartosc;
```

- Przejście do następnego elementu:

```
elem = elem->next;
```

Modyfikacja BUCKETSORT ma polegać na tym, aby algorytm sortował dla dowolnych danych wejściowych. Jednakowoż działa on bardzo w analogiczny sposób gdyż moja implementacja ten modyfikacji polega na zmianie numeracji, normalizacji kubełków.

Listing 14: Kod funkcji sortowania kubełkowego z modyfikacją

```

1 void BUCKETSORTMOD(float A[] , int n) {
2     float min = A[0];
3     float max = A[0];
4
5     for (int i = 1; i < n; i++) {
6         if (A[i] < min) {
7             minimum = A[i];
8         }
9         if (A[i] > max) {
10            max = A[i];
11        }
12    }
13
14    if (max == min) {
15        for (int i = 0; i < n; i++) {
16            A[i] = min;
17        }
18        return;
19    }
20 }
```

Początkowo znajdujemy wartość maksymalną oraz minimalną, aby mieć punkt odniesienia do skalowania wartości kubełków.

Listing 15: Kod funkcji sortowania kubełkowego z modyfikacją

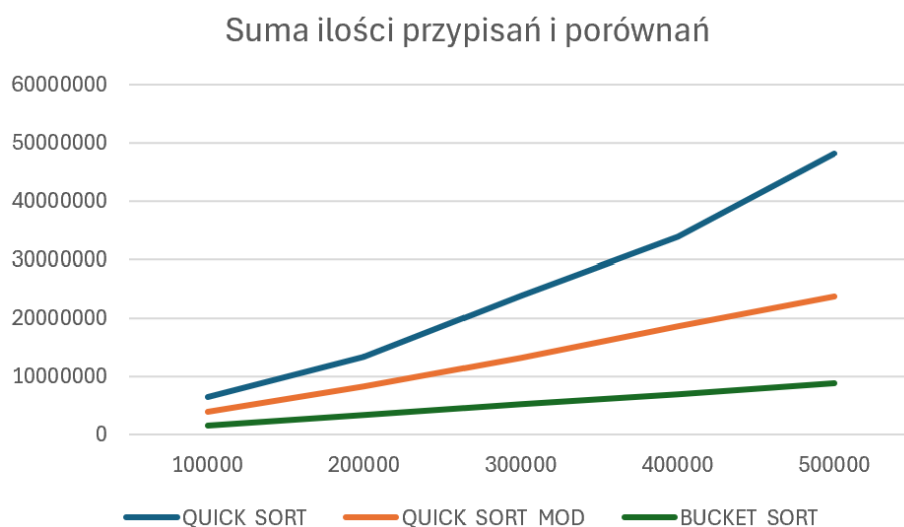
```

1 for (int i = 0; i < n; i++) {
2     int index = static_cast<int>
3     (n * (A[i] - min) / (max - min));
4
5     if (index == n) {
6         index = n - 1;
7     }
8 }
```

Dzięki odnalezieniu wartości maksymalnej oraz minimalnej jesteśmy w stanie zmodyfikować sprawdzaną wartość na liczbę z przedziału od 0 do 1. Pozostała część kodu jest identyczna jak przy normalnym sortowaniu kubełkowym.

## 4 Porównania algorytmów

### 4.1 QUICKSORT VS BUCKETSORT



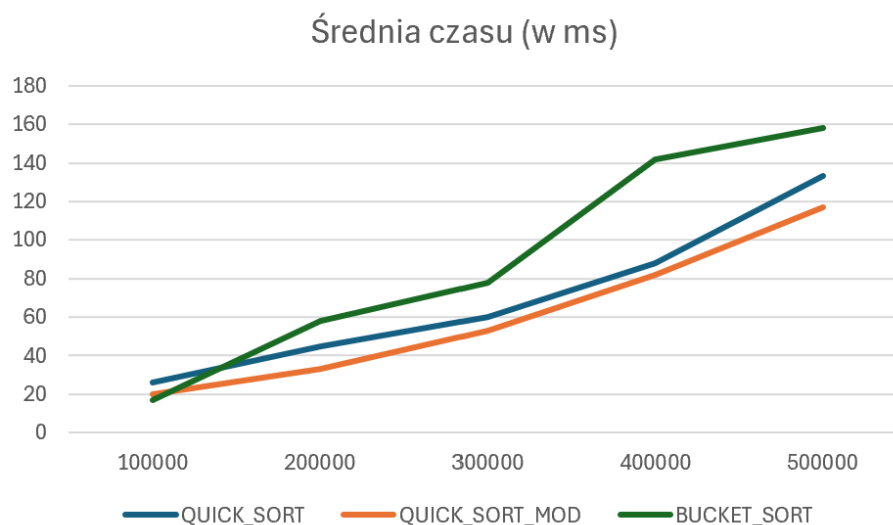
Rysunek 1: Wykres sumy przypisań i porównań w QUICKSORT, QUICKSORT MOD i BUCKETSORT.

| Ilość porównań |            |                |             |  |
|----------------|------------|----------------|-------------|--|
| Elementy       | QUICK_SORT | QUICK_SORT_MOD | BUCKET_SORT |  |
| 100000         | 1256117    | 988791         | 347003      |  |
| 200000         | 2623936    | 2075870        | 735463      |  |
| 300000         | 4680321    | 3388213        | 1134527     |  |
| 400000         | 6640204    | 4993907        | 1534451     |  |
| 500000         | 9431563    | 6383043        | 1934431     |  |

Rysunek 2: Tabela ilości porównań w QUICKSORT, QUICKSORT MOD i BUCKETSORT.

| Ilość przypisań |            |                |             |  |
|-----------------|------------|----------------|-------------|--|
| Elementy        | QUICK_SORT | QUICK_SORT_MOD | BUCKET_SORT |  |
| 100000          | 5173254    | 3035692        | 1270364     |  |
| 200000          | 10831668   | 6234602        | 2668745     |  |
| 300000          | 19255852   | 9797429        | 4068859     |  |
| 400000          | 27295286   | 13693328       | 5468878     |  |
| 500000          | 38660718   | 17309204       | 6868825     |  |

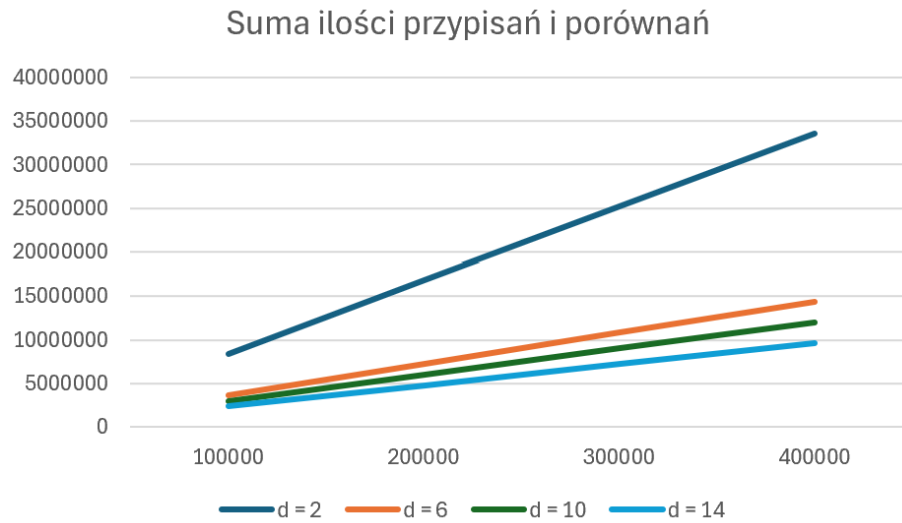
Rysunek 3: Tabela ilości przypisań w QUICKSORT, QUICKSORT MOD i BUCKETSORT.



Rysunek 4: Wykres czasu działania QUICKSORT, QUICKSORT MOD i BUCKETSORT.

Pod względem złożoności obliczeniowej, możemy powiedzieć, że najłżejszym algorytmem jest ewidentnie BUCKETSORT. Wykres wygląda jakby był liniowy i zgadza się to z założeniem teoretycznym jako, że ma on złożoność obliczeniową  $O(n)$ . Drugim poz względem złożoności będzie QUICKSORT MOD, który na w miarę wydłużania się listy, coraz bardziej oddalał się od normalnego QUICKSORTA. W tym przypadku również możemy wywnioskować złożoność, gdyż wykres sugeruje nam, że QUICKSORT będzie  $O(n^2)$ , a zmodyfikowany  $O(n \log n)$ , co również pokrywa się ze stroną teoretyczną. Jeśli jednak patrzymy na wydajność, złożoność czasową, to najkrócej działającym algoytmem jest zmodyfikowany QUICKSORT, a zdecydowanie najwolniejszym BUCKETSORT.

## 4.2 RADIXSORT a różne systemy liczbowe



Rysunek 5: Wykres sumy ilości przypisań i porównań RADIX SORTA.

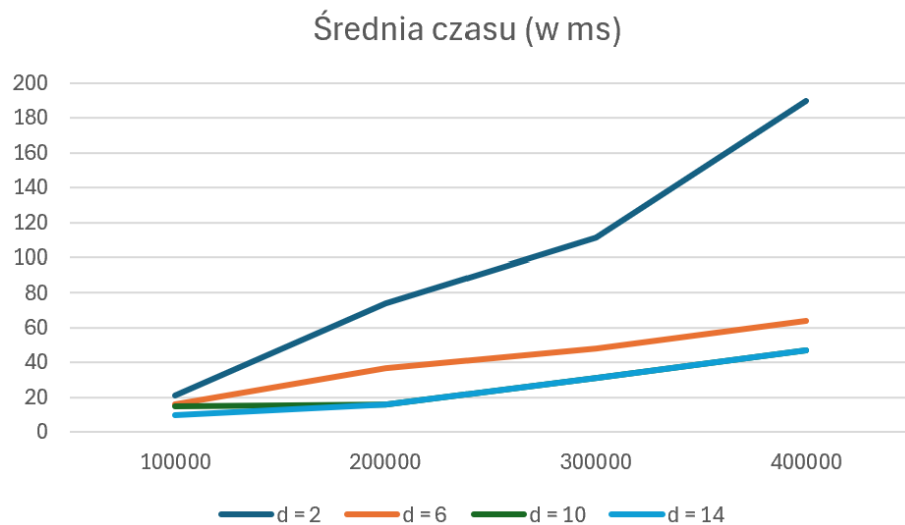
| Ilość porównań dla RADIX SORT |       |       |        |        |  |
|-------------------------------|-------|-------|--------|--------|--|
| Elementy                      | d = 2 | d = 6 | d = 10 | d = 14 |  |
| 100000                        | 23    | 11    | 10     | 9      |  |
| 200000                        | 23    | 11    | 10     | 9      |  |
| 300000                        | 23    | 11    | 10     | 9      |  |
| 400000                        | 23    | 11    | 10     | 9      |  |

Rysunek 6: Tabela ilości porównań RADIX SORTA.



| Ilość przypisań dla RADIX SORT |          |          |          |         |
|--------------------------------|----------|----------|----------|---------|
| Elementy                       | d = 2    | d = 6    | d = 10   | d = 14  |
| 100000                         | 8400080  | 3600060  | 3000071  | 2400074 |
| 200000                         | 16800080 | 7200060  | 6000071  | 4800074 |
| 300000                         | 25200080 | 10800060 | 9000071  | 7200074 |
| 400000                         | 33600080 | 14400060 | 12000071 | 9600074 |

Rysunek 7: Tabela ilości przypisań RADIX SORTA.



Rysunek 8: Wykres średniego czasu działania RADIX SORTA.

Porównania dla różnych systemów liczbowych dla algorytmu RADIX SORT pokazują, że im większą wartość przyjmuje nasze  $d$  (podstawa systemu liczbowego) to zarówno złożoność jak i czas jest coraz mniejsza/krótszy. Najdłuższym działaniem może pochwalić się system binarny, który znacznie odstaje od pozostałych pod względem złożoności jak i czasu. System dziesiętny oraz czternastkowy są bardzo blisko siebie, co mówi nam, że różnice się wypłaszczają, zmniejszają wraz z wzrostem  $d$ . Stąd najwydajniejszy RADIX SORT wyszedł dla  $d = 14$ , a najmniej optymalny dla  $d = 2$ .